

Product Requirements Document

Micro-Investment & Savings Platform for Gig Workers

PS Code: PS — 02 Track: FinTech / AI Agents Version: 2.0 (updated with AI-agent architecture, security/compliance, and India-specific free-tier features)

1. Problem Statement

Gig workers (delivery riders, cab drivers, freelancers, part-time platform workers) earn irregular, unpredictable income and have no access to employer-backed savings schemes (PF, gratuity, structured deductions). Existing investment products assume fixed monthly income and require manual, disciplined action — something most gig workers don't have the time, financial literacy, or bandwidth for.

Goal: Build an intelligent financial agent that:

- Analyses income patterns automatically
- Enables micro-savings without manual effort
- Recommends investment options based on behaviour and risk appetite
- Assists with basic tax tasks
- Makes financial planning passive, not a chore

Key Differentiator (pitch line):

Traditional finance apps: Transaction → Dashboard This platform: Transaction → Understanding → Prediction → Decision Support → Action

2. Target Users

Persona	Description	Key Pain Point
Delivery Partner	Daily/weekly payouts from platforms like Zomato, Swiggy, Blinkit	Income varies day to day, no savings habit
Cab/Auto Driver	Daily cash + digital earnings, fuel-heavy expenses	Hard to separate savings from working capital
Freelancer	Project-based, lumpy payments	Doesn't know how to plan for tax season
Part-time gig worker	Multiple small income sources	No single view of total earnings

Secondary users (future/B2B angle): Financial advisors, fintech companies, banks/NBFCs, insurance providers.

3. Goals & Success Metrics

Goal	Metric
Increase savings adoption	% of users with active auto-save rule
Reduce manual effort	Avg. manual transactions per user/month (target: near 0)
Improve financial literacy	% users who view/act on a recommendation
Tax readiness	% users with tax summary generated before filing season
Resilience	% users reaching emergency-fund milestone

4. Scope

4.1 In Scope (MVP)

- Income data ingestion (manual upload / CSV / mock UPI feed for demo)
- Income pattern analysis (moving average, variance, good-day/bad-day detection)
- Rule-based auto-savings engine (% of credit, round-off, surplus-based)
- Risk profiling (short questionnaire + behaviour signals)
- AI-driven investment recommendations (explained in plain language)
- Basic tax estimator (estimated liability, presumptive taxation hints for gig income)
- Emergency-fund tracker with progress bar
- Conversational AI financial agent (tool-calling, not free-form)
- Dashboard: earnings trend, savings growth, recommendations, tax summary

4.2 Out of Scope (for MVP)

- Real bank/UPI account linking (Account Aggregator integration) — stubbed/mock
 - Actual execution of investments (no real money movement)
 - Regulatory-grade tax filing (advisory only, not filing)
 - Real insurance policy issuance (informational suggestion only)
-

5. Functional Requirements

5.1 Income Analysis Engine

- Ingest transaction/income records (date, amount, source)
- Detect income frequency, average, volatility, peak/low-earning periods
- Classify days/weeks as high/low/irregular income

5.2 Auto Micro-Savings

- User sets a savings rule during onboarding (e.g., "save 10% of every credit" or "round up to nearest ₹50")
- System auto-computes and logs savings transactions (simulated ledger for MVP)
- User can pause/adjust rule anytime
- Savings goal tracker (e.g., emergency fund, festival fund, new laptop)

5.3 Risk Profiling & Recommendations

- Short onboarding questionnaire (risk appetite, goals, time horizon)
- Behavioural signals (income stability, savings consistency) refine the profile over time
- AI agent explains recommendations in simple language (e.g., "Since your income is stable, a Recurring Deposit or low-risk mutual fund fits you")
- Recommendations mapped to real product categories: RD, FD, liquid mutual funds, index funds, PPF/NPS (informational only — simulation, not execution)

5.4 Emergency Fund

- Target calculated from essential monthly expenses × selected number of months
- Current amount, target, and progress shown on dashboard
- Milestone notifications (25%, 50%, 100%)

5.5 Tax Assistant

- Estimate presumptive tax liability based on total logged income (Section 44ADA/44AD style estimation for freelancers/gig workers, shown as guidance only)
- Reminders for advance tax due dates
- Simple summary export (PDF/CSV) for the user's own reference

5.6 AI Financial Agent (conversational layer)

- Answers questions using the user's actual data: *"I earned ₹1,800 today, how much can I safely save?"*, *"Can I afford ₹2,000 investment this month?"*
- Agent does not perform calculations itself — it calls backend tools and explains the validated result (see Section 8: AI Architecture)

5.7 Dashboard

- Income trend chart
- Savings growth chart
- Emergency fund progress
- Active recommendations panel
- Tax summary card
- Quick access to AI agent chat

6. Non-Functional Requirements

- Usability: Must work with minimal manual input; mobile-first UI
- Performance: Dashboard load < 2s for typical data volumes
- Security: All financial data encrypted at rest (Supabase RLS + encrypted columns for sensitive fields); see Section 9 for full requirements
- Reliability: Auto-savings computation must be idempotent (no double-deduction on retry)
- Explainability: Every AI recommendation must show a plain-language reason, not just an output — numbers always come from deterministic calculation, never invented by the LLM

7. Proposed Tech Stack

Layer	Choice
Frontend	React / Next.js + TailwindCSS, Recharts for graphs

Backend	Node.js + Express
Database	Supabase (Postgres), pgvector optional for behaviour embeddings
Auth	Supabase Auth (email/OTP)
AI Layer	OpenRouter (LLM) with tool/function calling for recommendation reasoning + natural-language explanations
Automation	node-cron for scheduled auto-save runs
Hosting	Backend: Render/Railway · Frontend: Vercel
Data Input	CSV upload / manual entry for MVP; Account Aggregator (Setu sandbox) as stretch goal

8. AI Architecture (Agent Design Principle)

Rule: The LLM never performs financial calculations directly or silently executes an action. It orchestrates tools and explains results in plain language — all numbers come from deterministic rules/code.

User → AI Agent (LLM) → Financial Tools → Deterministic Calculation Engine → Result
→ AI Explanation

Exposed tools for the agent:

-
-
-
-
-
-
-

This separation matters for a hackathon pitch: it shows judges the system won't "hallucinate" a savings or tax number — the LLM only explains numbers a rules engine already validated.

9. Security & Compliance Requirements

- HTTPS everywhere
 - Encryption at rest (Supabase + encrypted sensitive columns)
 - Secure authentication (Supabase Auth, JWT session handling)
 - Input validation on all endpoints
 - API rate limiting
 - Audit logs for financial actions
 - Minimal data collection principle
 - Never store bank passwords, UPI PINs, card PINs, or OTPs
 - DPDP Act 2023 consent flow — explicit user consent screen before collecting financial data, with a plain-language privacy notice (strong point for SIH-style judging)
 - Clear labeling: distinguish financial education vs. recommendation vs. actual transaction — MVP only simulates the last one
-

10. High-Level Data Model

users: id, name, income_type, risk_profile, onboarding_answers, preferred_language

transactions: id, user_id, amount, type (credit/debit), source, category, timestamp

savings_rules: id, user_id, rule_type, value, active

savings_ledger: id, user_id, amount, triggered_by_transaction_id, timestamp

recommendations: id, user_id, product_type, reason_text, generated_at

tax_estimates: id, user_id, period, estimated_liability, generated_at

goals: id, user_id, goal_type, target_amount, current_amount, deadline

streaks: id, user_id, current_streak, longest_streak, last_saved_date

11. Core User Flow

1. User signs up → DPDP consent screen → completes short risk/income questionnaire
 2. Uploads/logs income (CSV, manual entry, or OCR screenshot import for MVP)
 3. System analyses pattern → shows income trend
 4. User sets a savings rule → auto-savings starts running on new income entries
 5. AI agent generates a recommendation with explanation
 6. Dashboard shows savings growth, emergency-fund progress, and tax estimate over time
 7. User can chat with the AI agent for what-if questions, adjust rules, or view updated recommendations anytime
-

12. India-Specific Feature Additions (Free-Tier Feasible)

These extend the "protect and reach every gig worker" angle and are achievable within a hackathon budget:

Feature	Why it matters	Free-tier approach
WhatsApp-based interaction	Most gig workers already use WhatsApp, may avoid installing a new app	Twilio WhatsApp Sandbox for demo (free for testing; production WhatsApp Cloud API is no longer flat-rate free, so keep this to demo scope)
Voice / vernacular support	Target users may be more comfortable speaking than typing English	Bhashini (Govt of India's open, free language API) for Hindi/regional speech-to-text, text-to-speech, and translation
Offline-first design	Delivery/cab drivers have patchy connectivity	PWA + Service Workers + IndexedDB — no paid service, just architecture
Micro-insurance bundling	Physically risky jobs; natural extension of the "protect" pillar	Informational/simulated suggestion only in MVP — no real insurance API needed

DPDP Act consent flow	India's data protection law; strong compliance signal to judges	Just a consent UI + logged timestamp — no cost
Gamified savings streaks	Motivation for irregular-income users	Pure app logic + DB table — no cost
OCR income import	Real bank/platform API access is hard to get; screenshot import feels "smart"	Tesseract.js (open-source, runs client-side or in Node, no API key needed)

13. Killer Demo Flow (for judging)

1. Create a fictional delivery-worker profile
 2. Upload sample income/expense data (CSV or OCR screenshot)
 3. Show income volatility chart + AI-generated insight ("Your income is 18% higher on weekends")
 4. Trigger an auto-savings rule and show the ledger update
 5. Show emergency-fund progress
 6. Generate a risk profile → show investment-category suggestion with explanation
 7. Generate a tax-reserve estimate
 8. Wow moment: Ask the AI agent — *"I need ₹8,000 for a bike repair next month, can I afford it?"* — agent uses projected cash flow, expenses, savings, and goals to explain the trade-off instead of a generic chatbot answer
-

14. Risks & Assumptions

Risk	Mitigation
No real bank data access in MVP	Use realistic mock/CSV data or OCR-imported screenshots for demo
Users distrust AI investment advice	Always show reasoning, avoid definitive "buy" language, mark as informational
Tax estimation inaccuracy	Clearly label as "estimate, not filing advice"
Low engagement without real money	Frame MVP as a simulation/sandbox savings tracker for demo purposes
WhatsApp production costs	Keep WhatsApp interaction scoped to Twilio Sandbox for the demo; note production integration as a Phase 4 item

15. Milestones (Hackathon Build Plan)

Phase	Deliverable
Hour 0–4	Auth, DPDP consent screen, DB schema, CSV/OCR upload, income parsing
Hour 4–10	Auto-savings rule engine + ledger + streaks
Hour 10–16	Risk profiling + AI agent with tool-calling integration
Hour 16–20	Tax estimator + emergency fund + dashboard charts
Hour 20–24	Polish UI, Bhashini voice input (stretch), demo data, pitch deck

16. Product Roadmap (Beyond Hackathon)

- Phase 1: Income tracking, expense tracking, dashboard, micro-savings, emergency fund
 - Phase 2: AI forecasting, risk profiling, investment recommendations, tax assistant
 - Phase 3: AI financial agent, what-if simulator, personalized notifications, WhatsApp/voice access
 - Phase 4: Authorized bank/UPI integrations, automated savings, investment execution, tax filing integration, real insurance partnerships
-

17. Open Questions

- Should auto-savings be simulated (in-app ledger) or actually integrated with a real savings account API?
- Should tax estimation cover only presumptive taxation, or also GST implications for freelancers?
- Is WhatsApp/voice interaction a must-have for the demo, or a stretch goal after the core dashboard works?